RAG Frameworks — Hands-on Exercises

LangChain • LlamaIndex • Haystack | 90–120 minutes | PyCharm + Python

Goal: Use three different Python frameworks to search the same fictional science-museum documents. Focus on loading, chunking, embedding and retrieval. No React UI or paid language-model API is needed.

Provided data

Document	Contains
Visitor_Guide.txt	Opening hours, admission, tickets, visitor facilities and photography
Workshops_and_Events.txt	Workshop schedules, age limits, pricing, rules and cancellations

Common setup

- Create a new practice folder in PyCharm. Put both supplied .txt files inside Documents/.
- Reuse your existing examples as references, but write one new Python script for each framework.
- Use local all-MiniLM-L6-v2 embeddings in every exercise. Stop at retrieval: do not call OpenAI.
- Print the retrieved chunk text and the source filename for each search.

Exercise 1 — LangChain (30 minutes)

- Create 01_langchain_museum.py. Read both .txt files and create LangChain Documents.
- Split text into chunks (about 150 tokens, 20-token overlap).
- Generate local embeddings and store chunks in ChromaDB collection museum_langchain.
- Accept a question and print the top 2 matching chunks, source files and distances.

Exercise 2 — LlamaIndex (30 minutes)

- Create 02_llamaindex_museum.py. Load both files with SimpleDirectoryReader.
- Create Nodes using SentenceSplitter (about 150 tokens, 20-token overlap).
- Build a VectorStoreIndex using MiniLM and a separate ChromaDB collection: museum_llamaindex.
- Use a retriever to display the top 2 matching nodes and their source files.

Exercise 3 — Haystack (30 minutes)

- Create 03_haystack_museum.py. Load the same two documents into Haystack Document objects.
- Build an indexing pipeline: DocumentSplitter → DocumentEmbedder → DocumentWriter.
- Use InMemoryDocumentStore and split by 100 words with 15-word overlap.
- Build a query pipeline: TextEmbedder → InMemoryEmbeddingRetriever, then print the top 2 chunks.

Test all three implementations

1. When does the museum close on Thursdays?
2. What is the adult admission ticket price?
3. What age group can join Robotics Starter Lab?
4. When does Space Explorers take place?
5. What happens if the museum cancels a workshop?
6. Is free parking available at the museum?

Observe: The last question has no stated answer in the supplied documents. The retrievers may still return chunks; retrieval alone cannot determine whether an answer exists.

Submission and comparison

- Submit three Python scripts and one screenshot of search results from each.
- Complete this comparison: Which framework felt simplest? How did their APIs differ? Did all three retrieve useful evidence for the same questions?

Optional extension: In Haystack, add separate Visitor and Workshop routes. Keep this optional; the three core retrieval exercises are the required work.